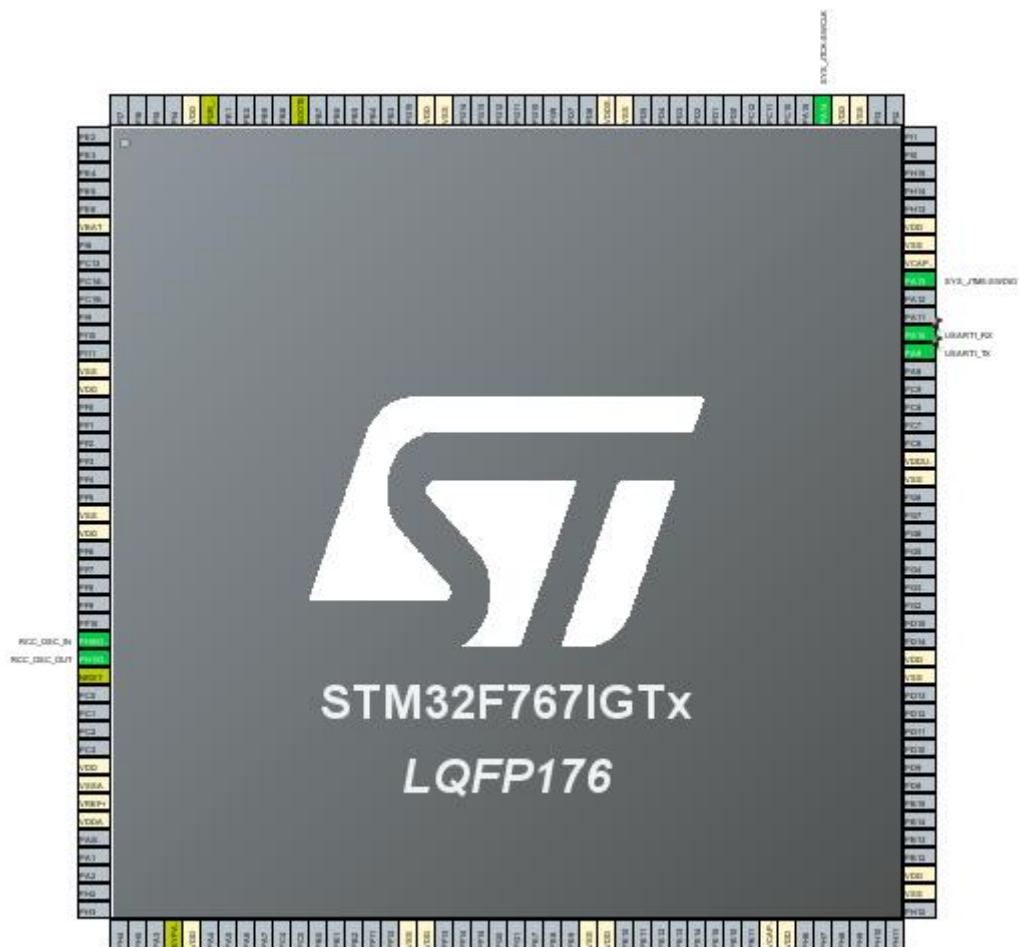


嵌入式大作业报告



完成人：刘志鸿

宋文博

日期：2025. 6. 15

一、大作业选题

利用 CUBE AI 在 STM32F767IGT6 开发板上部署手写识别系统。

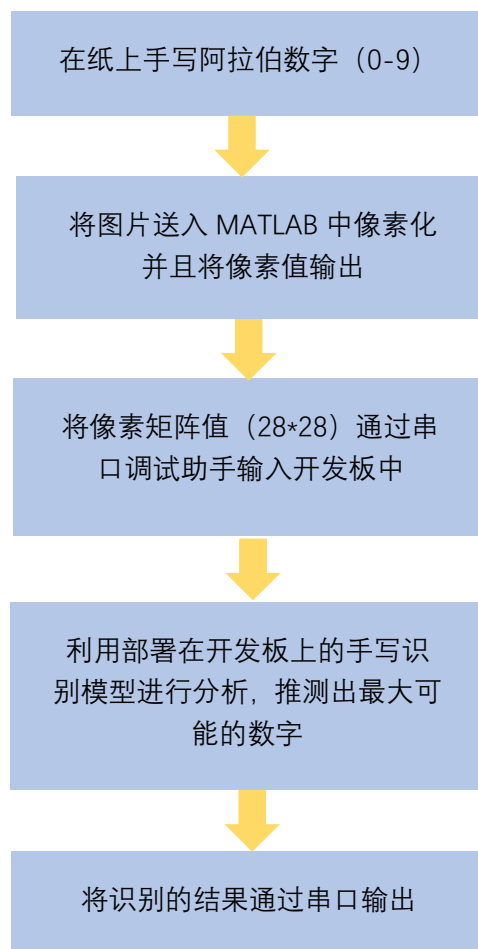
要求：通过串口、SD 卡、USB 口、以太网口或自带的触摸屏输入图像数据，识别结果通过串口或触摸屏输出。

二、硬件与软件平台

硬件：STM32cubeMX

软件：Python、Keil5、MATLABR2024a

三、大致实现流程框架图



四、利用 Python 训练手写识别模型

4.1 模型训练说明

利用循环神经网络中的长短时记忆网络 (LSTM) 模型对 MNIST 手写数字数据集中的手写数字图片进行识别分类进行分类，将图像转化为可以处理的“序

列数据”。设计 LSTM 网络结构、训练模型以及导出 TensorFlow Lite 模型，为后续嵌入式设备（stm32F7 开发板）部署神经网络模型做准备工作。

4.2 数据集介绍

1.数据来源

使用经典的 MNIST 数据集，包含 70,000 张 28×28 像素的灰度手写数字图片，其中训练集 60,000 张，测试集 10,000 张。数据标签为 0 到 9 共 10 类数字。

2.图像本质与灰度特性

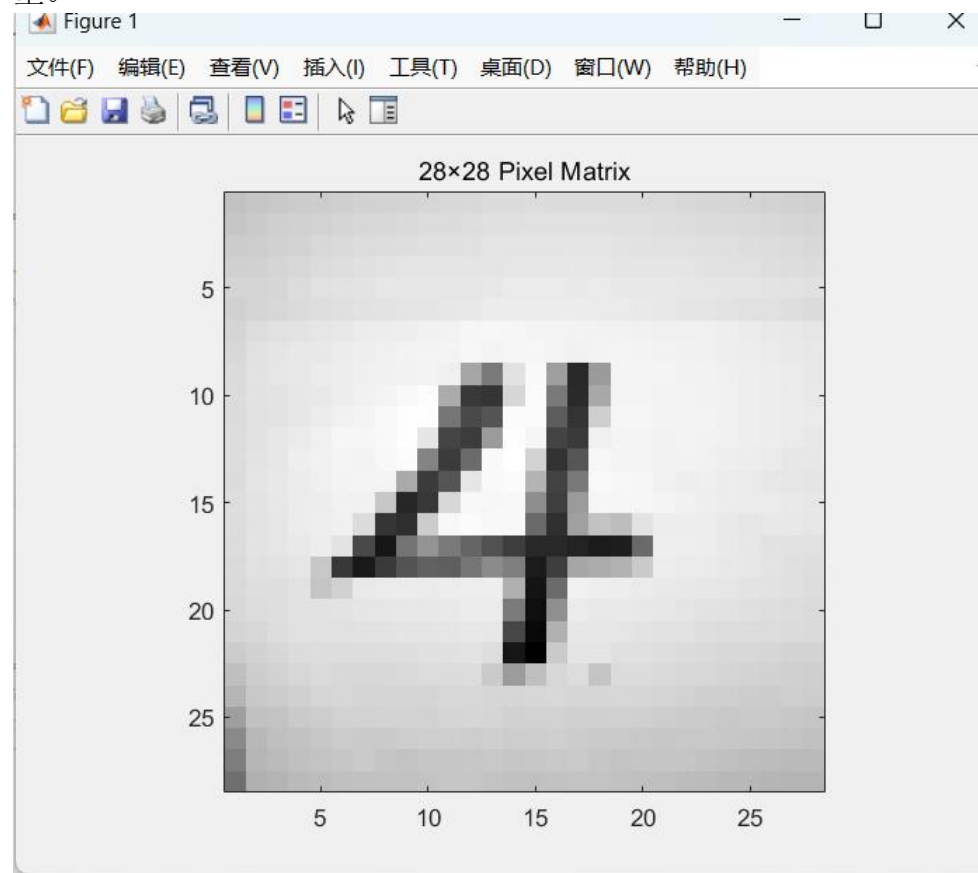
图像为单通道灰度图像，像素值为 0–255 的整数，表示图像的亮度

0 表示完全黑色（笔画）

255 表示完全白色（背景）

中间值表示不同灰度级别（笔画边缘、书写强弱等）

因此，图像是先通过灰度化处理（黑白处理）后编码为灰度像素值的二维张量。



3.数据预处理

为了适应神经网络的数值计算，所有像素值被归一化为浮点数范围 $[0.0, 1.0]$ ：

```
# 归一化处理
train_images = train_images.astype('float32') / 255.0
test_images = test_images.astype('float32') / 255.0
```

此操作将：0 转为 0.0，代表最黑；255 转为 1.0，代表最白；中间值为灰色，对应笔迹模糊、过渡区域

最终，模型处理的是归一化后的图像浮点矩阵，即每张图为一个 28×28 的二维浮点数组。

一、模型结构设计

1.模型概述

输入层：形状为 (28, 28)，即每张图片被视为 28 个时间步，每步 28 维特征的序列数据。

LSTM 层：包含 20 个神经元，激活函数为 tanh，设置 return_sequences=True，以输出序列的全部时间步信息。

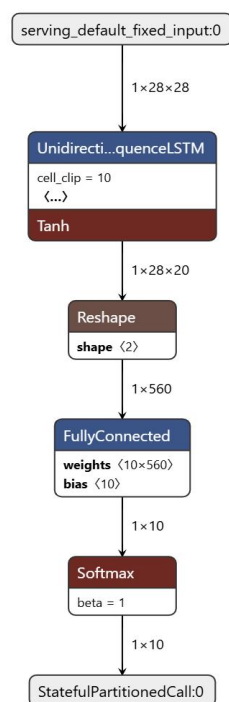
重塑层（Reshape）：将 LSTM 输出的 (28, 20) 张量展开为 (560,) 的一维向量，供后续全连接层使用。

全连接层（Dense）：输出 10 个类别的概率，激活函数为 softmax。

2.模型结构代码

```
# 输入形状为 (batch_size, 28, 28)，每行作为一个时间步
model = models.Sequential([
    layers.Input(shape=(28, 28), name='fixed_input'),
    layers.LSTM(20, activation='tanh', return_sequences=True, name='LSTM'),
    layers.Reshape((28 * 20,), name='reshape'),
    layers.Dense(10, activation='softmax', name='output')
])
```

3.模型可视化框图



二、训练过程

```
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

print("开始训练 LSTM 模型...")
history = model.fit(train_images, train_labels,
                    epochs=10,
                    batch_size=64,
                    validation_split=0.1,
                    verbose=1)
```

三、评估指标

accuracy: 评估模型好坏的指标

批大小 (Batch Size): 64

优化器: Adam (自适应优化器, 效率高, 学习速度快)

损失函数: 稀疏类别交叉熵 (sparse_categorical_crossentropy) 用于整数标签的分类问题

训练轮数 (Epochs): 10

验证集: 使用训练集的 10% 作为验证数据, 用于监控过拟合和模型泛化能力。

四、模型导出与轻量化部署

```
# 转换为 TFLite 模型
converter = tf.lite.TFLiteConverter.from_keras_model(model)
tflite_model = converter.convert()
with open('lstm_mnist_model.tflite', 'wb') as f:
    f.write(tflite_model)
print("模型已保存为 lstm_mnist_model.tflite")
```

模型成功保存为 Keras 格式的 lstm_mnist_model.h5。

使用 TensorFlow Lite 转换器将模型转换为 lstm_mnist_model.tflite，便于部署在嵌入式设备上。

五、MATLAB 实现图像预处理

我们使用 MATLAB 编写的图像处理脚本，将手写测试图片转化为模型可接受的 28×28 灰度像素值，用作串口调试助手的测试数据。

5.1 测试图片



5.2 实现流程

1. 读取图片并转换为灰度图像

使用 `imread()` 函数读取一张 JPG 格式的图片，输入图像可以是彩色或灰度图。文件名 'test.jpg' 可以替换为用户指定的图像文件路径。如果图像是 RGB 彩色图，调用 `rgb2gray()` 将其转换为灰度图。

```

% 读取图片
img = imread('test.jpg'); % 替换为你的图片路径

% 灰度化（如果是彩色图像）
if size(img, 3) == 3
    img_gray = rgb2gray(img);
else
    img_gray = img;
end

% 将图像大小调整为 28*28 的整数倍（例如：784x784）
[rows, cols] = size(img_gray);
block_size = 28;

```

2. 裁剪图像，使其行列都是 28 的整数倍

首先获取图像尺寸，并设定我们要划分成的块数为 28×28 ；接着为保证图像可以被均匀切割成 28×28 的小块，使用 `floor()` 将原图大小裁剪为最接近的 28 的倍数；取 `img_cropped` 为裁剪后的图像，避免因无法整除而出现边缘块尺寸不一的问题。

```

% 计算需要裁剪到的尺寸
new_rows = floor(rows / block_size) * block_size;
new_cols = floor(cols / block_size) * block_size;
img_cropped = img_gray(1:new_rows, 1:new_cols);

```

3. 定义每个小块的尺寸（每块压缩为一个像素）

```

% 每块的像素大小
tile_rows = new_rows / block_size;
tile_cols = new_cols / block_size;

```

4. 将每个小块区域平均为一个像素灰度值

使用嵌套 `for` 循环逐个提取图像中的小块；每个小块内像素值通过 `mean()` 求平均，形成一个代表块亮度的单一值；终得到一个 28×28 的像素平均值矩阵 `pixel_matrix`


```

% 分块并计算平均像素（取整）
for i = 1:block_size
    for j = 1:block_size
        row_start = (i-1)*tile_rows + 1;
        row_end   = i*tile_rows;
        col_start = (j-1)*tile_cols + 1;
        col_end   = j*tile_cols;

        block = img_cropped(row_start:row_end, col_start:col_end);
        avg_pixel = mean(block(:));
        pixel_matrix(i, j) = uint8(avg_pixel); % 转为整数
    end
end

```

5. 输出像素矩阵，以便输入进串口调试助手

```

% 输出28x28像素矩阵，每个值空格隔开
for i = 1:block_size
    for j = 1:block_size
        fprintf('%3d ', pixel_matrix(i, j));
    end
    fprintf('\n');
end

```

5.3 Matlab 输出像素数据

```

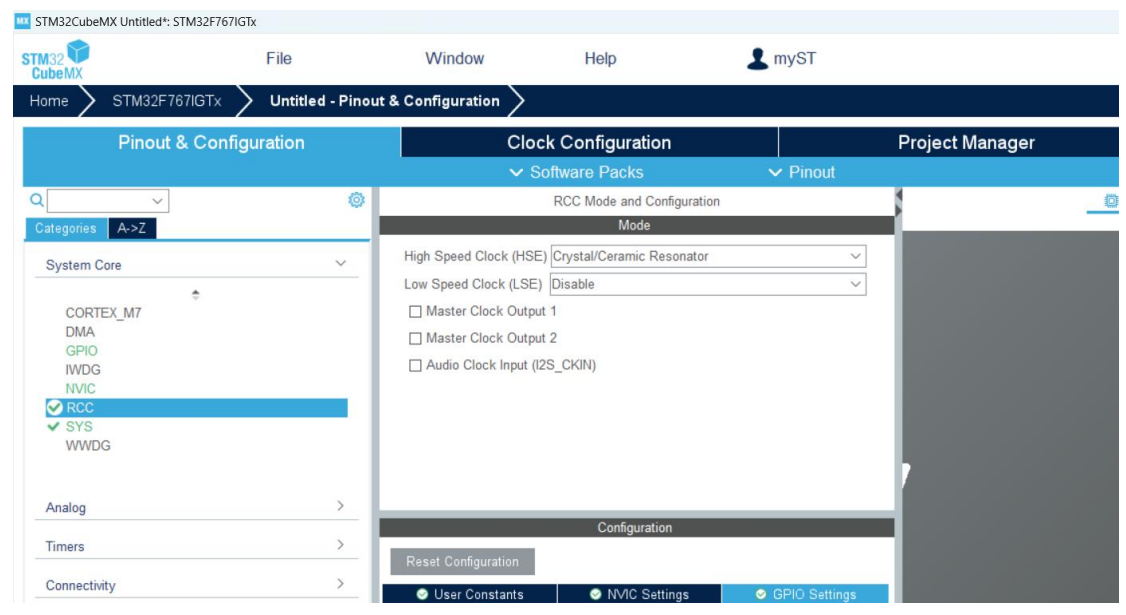
>> untitled2|
194 194 193 192 192 192 191 190 190 189 188 187 187 186 185 184 183 182 181 180 179 179 177 177 176 175 174 174
196 195 194 193 193 192 192 191 190 189 189 188 187 186 185 185 184 183 182 181 179 179 178 177 177 175 174 174
195 195 195 193 193 193 193 192 191 191 190 190 188 187 187 185 185 184 182 181 180 179 178 177 177 175 174 173
195 196 195 195 194 194 193 192 191 190 190 190 189 188 186 186 184 183 183 181 180 180 179 178 176 175 174 173
195 195 194 194 194 194 191 186 188 187 183 165 156 143 138 131 117 99 167 180 179 178 177 176 175 174 173
195 194 194 194 194 193 193 103 22 32 29 24 31 39 49 52 52 63 61 160 180 178 178 178 176 174 174 173
194 194 193 194 194 193 193 109 19 33 157 178 186 187 186 185 183 181 181 180 180 177 177 177 174 174 175 174
193 193 192 192 193 193 192 154 23 43 188 189 188 188 186 185 184 183 181 180 180 178 177 176 175 175 175 174
193 193 193 193 192 192 191 150 19 45 187 189 188 188 187 186 184 183 181 180 180 178 178 178 177 175 174
193 193 193 192 192 192 191 122 22 57 186 188 188 187 186 185 184 183 182 181 181 180 179 179 177 176 175 174
193 192 192 192 191 191 189 102 15 66 186 187 188 187 186 185 184 183 182 182 181 181 180 179 177 176 175 174
191 192 191 191 190 190 188 70 14 69 186 187 187 186 185 184 184 183 182 182 181 181 180 178 177 177 175 175
191 191 191 190 189 189 188 60 14 53 115 92 77 68 68 79 103 143 178 180 181 180 179 179 178 177 176 174
191 190 190 189 189 187 187 77 15 17 32 62 82 88 86 59 19 14 34 94 166 180 179 178 177 177 175 174
190 189 188 188 188 188 188 130 92 149 181 186 186 185 185 184 172 111 24 11 27 149 178 178 177 176 175 174
189 189 188 188 188 188 188 187 187 186 187 187 186 185 185 184 183 182 163 39 10 49 176 177 177 176 175 174
187 188 188 188 188 187 187 187 187 186 186 186 185 184 184 183 183 182 181 151 17 10 144 177 176 175 174 173
186 186 187 187 186 186 186 186 185 185 185 184 184 183 183 182 182 181 178 59 10 142 176 175 175 174 173
183 183 184 185 184 184 185 184 185 184 184 184 183 183 183 182 182 181 180 171 31 11 161 175 174 173 173 173
182 182 183 184 184 184 184 182 182 183 183 183 182 182 182 182 181 180 146 51 12 60 174 174 174 174 173 172
182 182 182 183 183 182 157 52 53 111 152 177 180 180 180 179 143 66 14 11 9 111 174 173 173 173 172 171
182 182 183 183 183 182 168 40 7 7 9 16 47 69 64 32 11 10 10 9 39 151 173 173 172 171 171 170
182 183 182 181 181 180 179 170 70 5 6 8 9 9 8 8 8 8 26 112 172 174 173 172 171 170 170 170
181 181 180 179 178 177 177 176 175 118 27 6 8 15 35 67 94 133 167 173 173 173 172 171 170 170 169 168
179 179 179 178 177 177 176 175 175 174 172 162 166 172 175 174 173 173 173 172 172 172 171 170 170 169 168 168
177 177 177 176 175 175 175 174 173 173 172 173 173 173 174 173 173 172 173 172 171 171 170 169 168 167 166 166
175 175 175 174 174 173 174 174 173 172 172 172 172 173 173 172 171 171 171 170 170 169 168 167 167 166 165
173 173 172 172 172 172 172 172 172 172 172 172 172 172 171 170 169 169 169 169 169 168 167 166 165 165

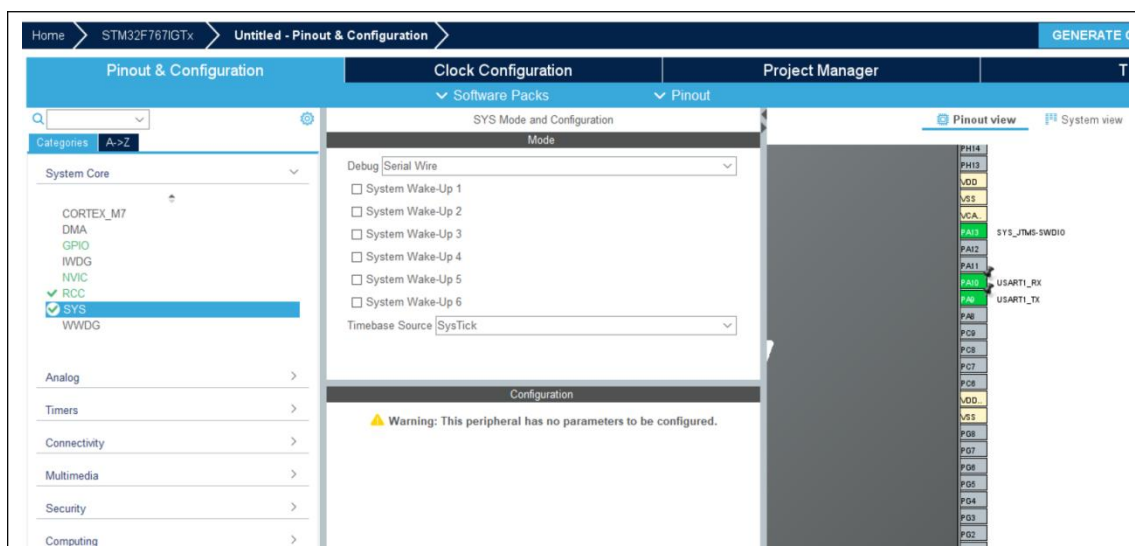
```

六、在 STM32cubeMX 上部署硬件设置

6.1 时钟源时钟树设置

时钟源选择 HSE 外部晶振时钟，时钟树按如下配置

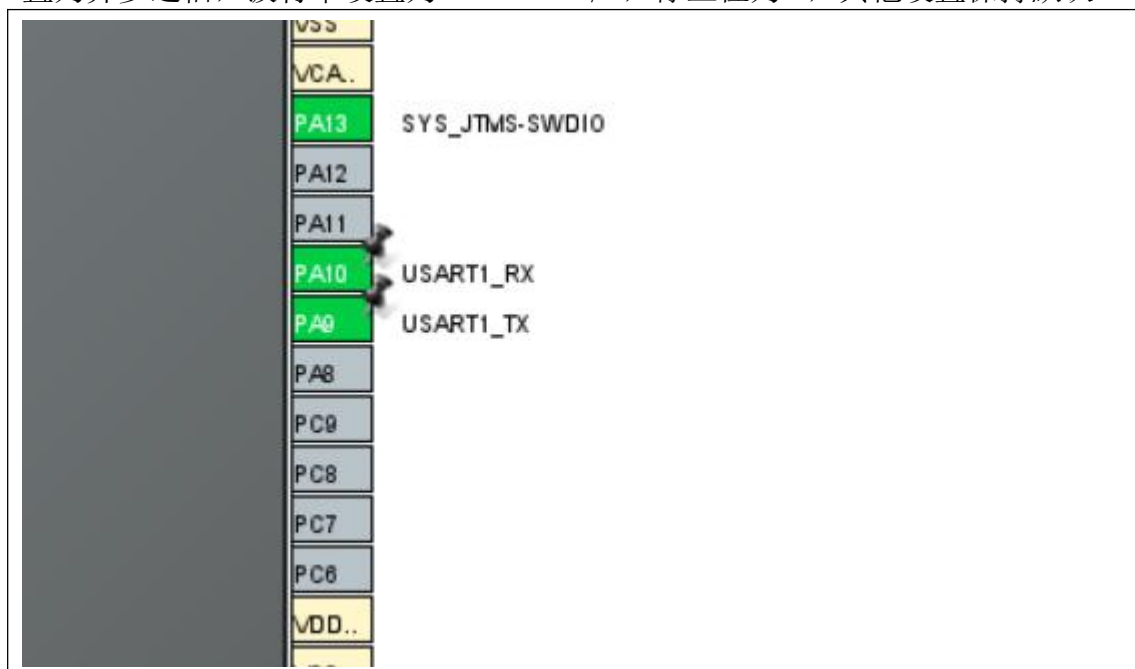




6.2 GPIO 配置



根据原理图将 PA9、PA10 管脚分别设置为 USART1_TX 和 USART_RX，管脚配置为异步通信，波特率设置为 9600 bits/s，停止位为 1，其他设置保持默认。



USART1 Mode and Configuration

Mode

Mode

Asynchronous

Hardware Flow Control (RS232)

Disable

☐ Hardware Flow Control (RS485)

Configuration

Reset Configuration

Configuration

Reset Configuration

Parameter Settings

User Constants

NVIC Settings

DMA Settings

GPIO Settings

Configure the below parameters :

Search (Ctrl+F)

Basic Parameters

Baud Rate

9600 Bits/s

Word Length

8 Bits (including Parity)

Parity

None

Stop Bits

1

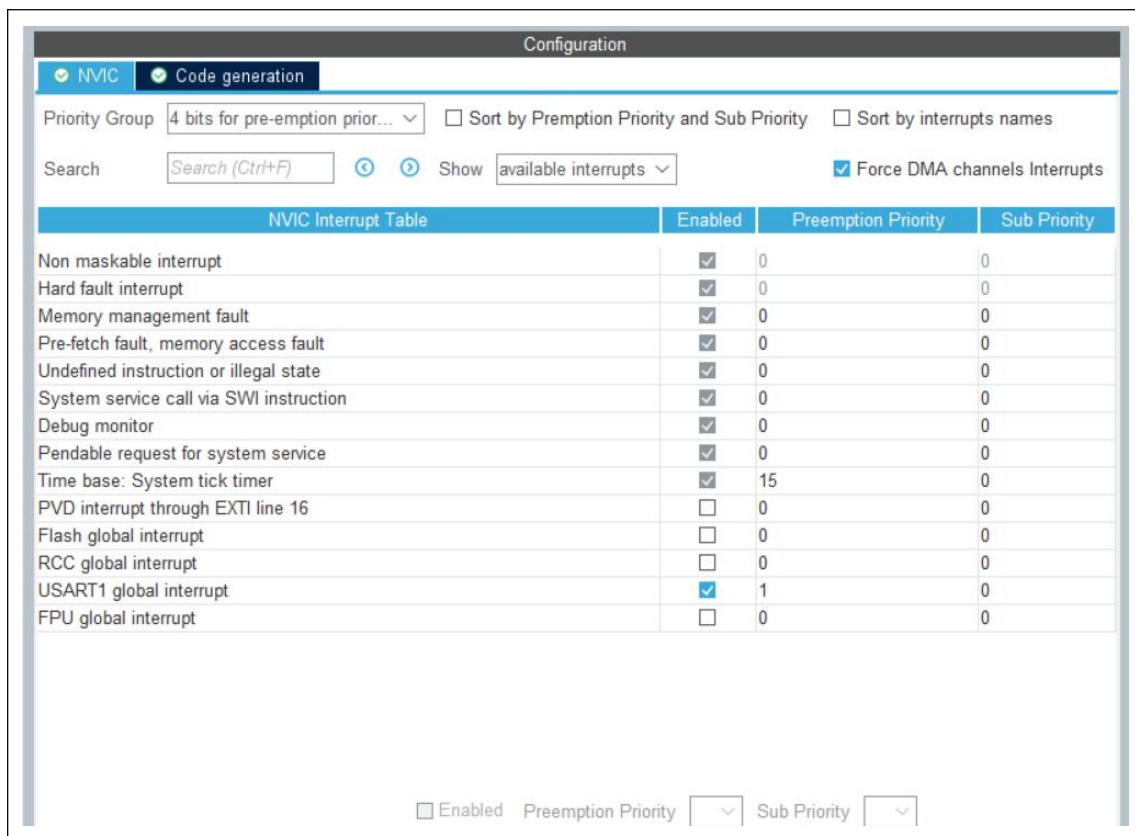
Advanced Parameters

Data Direction

Receive and Transmit

6.3 NVIC 配置

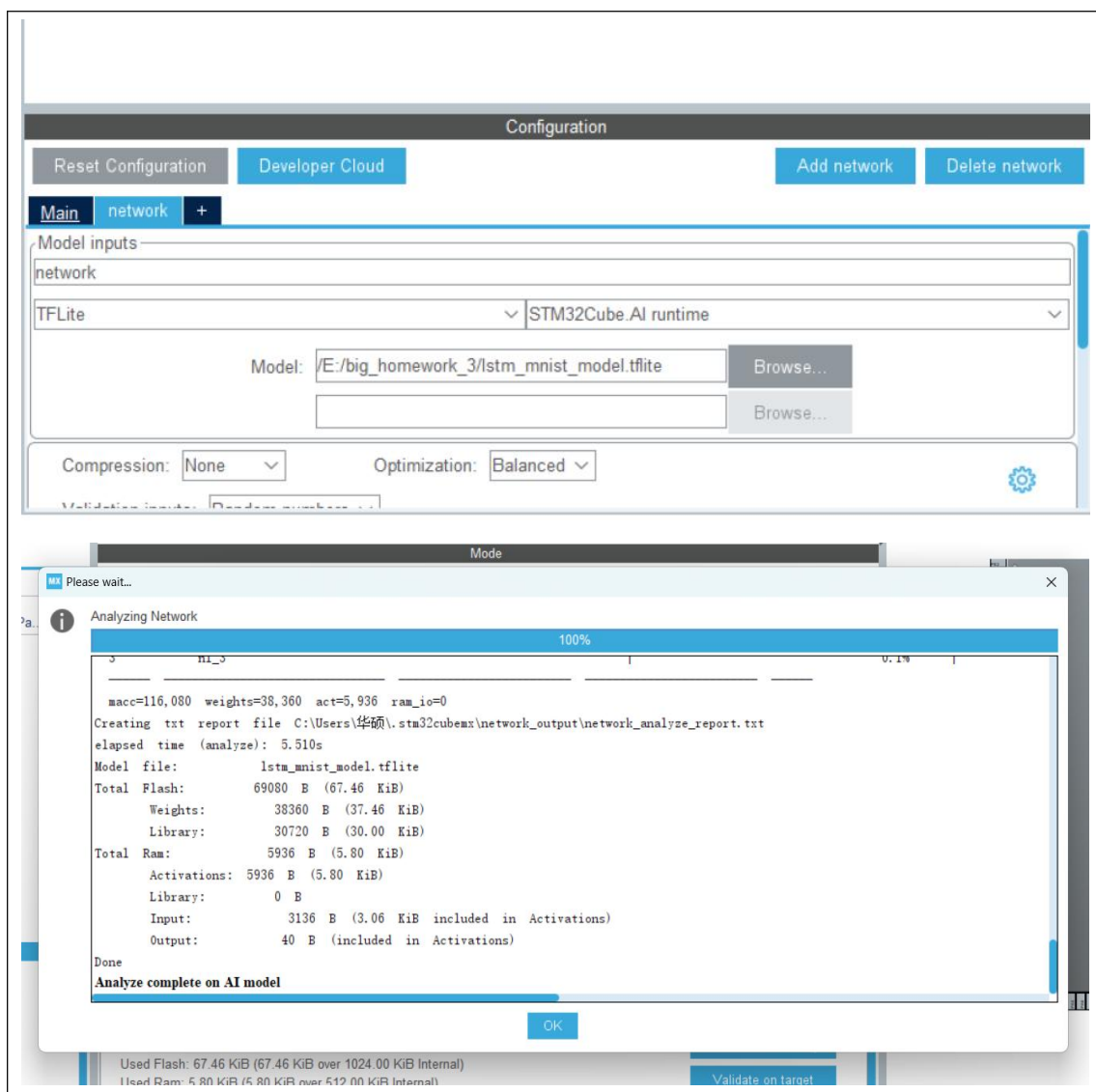
整个程序只需要使能串口 USART1 中断即可，中断优先级设为 1，其余不用动。



6.4 cube ai 配置

在 Middleware and software packs 选项中，打开 x-cube-ai，添加 ai 模型，添加完成后，可以点击 analyze 对模型 进行分析，查看模型的大小及占用的资源，如果占用的资源超出芯片本身的容量，将出现红色警示。





七、Keil5 中进行 C 语言编程

7.1 串口输入处理

这里其实参考了课上讲过的串口接收不定长字符串的方法，因为每个像素值在 0-255 之间，而输入的时候是将一个包含了 784 个像素，每个像素之间用空格隔开的字符串输入的。我们在 while 循环使用 HAL_UARTEx_ReceiveToIdle_IT 函数实现这一功能，当总线空闲的时候一次性读取完整整个字符串，避免使用 HAL_UART_Receive_IT 还需要对输入进来的数据逐个判定。然后将输入进来的字符串用 strtok 函数切割开，切割符号为 “ ”（空格）（此处对应 MATLAB 实现图片像素化的部分时的输出结果），再利用 atof 转换为 float 型数按照顺序送进 ai 的输入数组中。

另外，我们识别的时候是白底黑字，白色对应像素值是 255，黑色对应的是 0，所以不需要反转，同时做归一化处理，需要除以 255 将值限定在[0,1]之间。


```

while (1)
{
    if(My_Uart_Rx.RxFlag == 1)//接收完一帧数据
    {
        printf("The received strings are : %s\r\n",My_Uart_Rx.RxBuf);
        printf("UART Received %d Bytes\r\n",My_Uart_Rx.RxLen);
        My_Uart_Rx.RxFlag = 0;
        char *p;
        uint16_t i =0;
        p = strtok((char *)My_Uart_Rx.RxBuf, " "); //切割数据
        printf("UART Received %d Bytes\r\n",My_Uart_Rx.RxLen);
        printf("The received strings are : %s\r\n",My_Uart_Rx.RxBuf);
        while(( p!=NULL ))//数组范围: 0-783
        {
            aiInData[i] = atof(p)/255; //ASCII 转化为浮点型
            p = strtok(NULL, " ");
            i++;
            if( p==NULL )
            {
                in_data_size = i;
                printf("\r\nYou Have Received %d Numbers\r\n",in_data_size);//检查是否为28*28
                AI_Run(aiInData, aiOutData);
                memset(My_Uart_Rx.RxBuf, 0,My_Uart_Rx.RxLen); //清空缓存, 准备下一次接收
            }
        }
    }
}

```

7.2 引入需要的宏定义、结构体和全局变量

```

#define RX_MAXLEN 5000 //最大接收数据长度
typedef struct
{
    uint8_t RxBuf[RX_MAXLEN]; //接收缓存
    uint16_t RxCnt; //接收数据计数
    uint16_t RxLen; //接收数据长度
    uint8_t RxStart; //开始接收标志
    uint8_t RxFlag; //一帧数据接收完成标志
}Uart_Rx_Tpye_t;

Uart_Rx_Tpye_t My_Uart_Rx;

uint16_t in_data_size;
//模型网络
ai_handle network;
//模型输入数据
float aiInData[AI_NETWORK_IN_1_SIZE] = {0};
//模型输出数据
float aiOutData[AI_NETWORK_OUT_1_SIZE];
ai_u8 activations[AI_NETWORK_DATA_ACTIVATIONS_SIZE];

ai_buffer * ai_input;
ai_buffer * ai_output;
//全局静态函数
static void AI_Init(void);
static void AI_Run(float *pIn, float *pOut);

```

7.3 模型运行

将处理好的数据作为形参运行函数 AI_Run(float *pIn, float *pOut), 调用模

型后得到输出结果，在串口中输出 0-9 一共 10 个数字的对应识别概率，选择识别概率最大认为是识别结果。

```
static void AI_Run(float *pIn, float *pOut)
{
    int count = 0;
    float max = 0;
    ai_i32 batch;
    ai_error err;

    // 更新io处理程序与数据负载
    ai_input[0].data = AI_HANDLE_PTR(pIn);
    ai_output[0].data = AI_HANDLE_PTR(pOut);

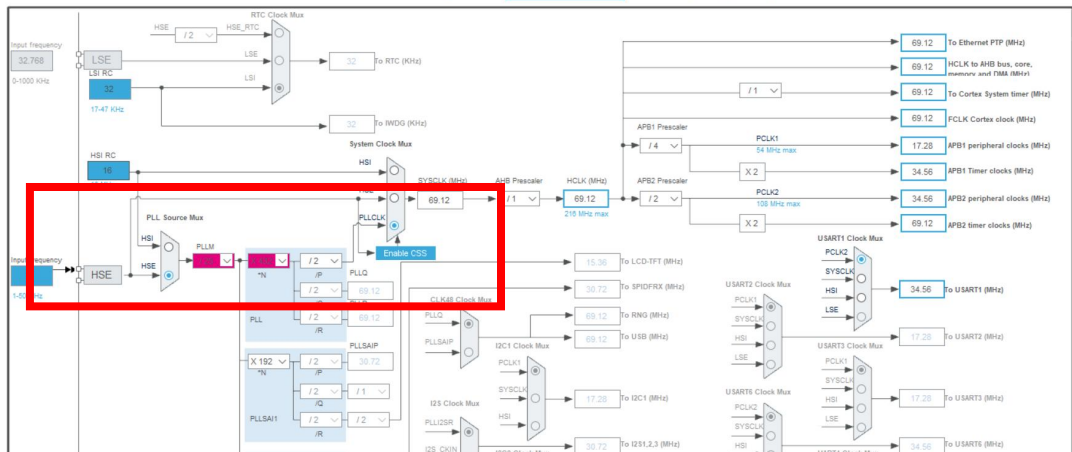
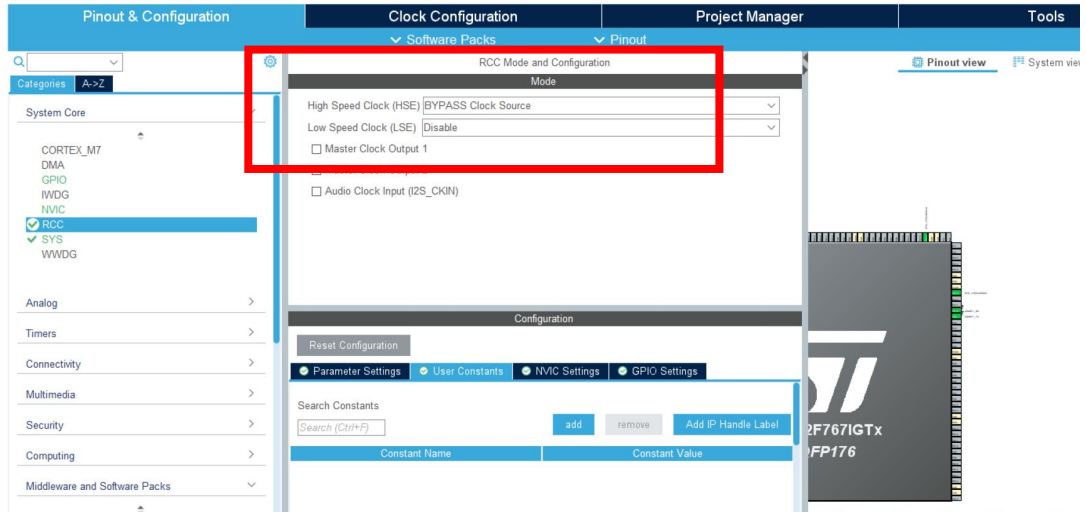
    batch = ai_network_run(network, ai_input, ai_output);
    if (batch != 1)
    {
        err = ai_network_get_error(network);
        printf("AI ai_network_run error - type=%d code=%d\r\n", err.type, err.code);
        Error_Handler();
    }
    for (uint32_t i = 0; i < AI_NETWORK_OUT_1_SIZE; i++)
    {
        printf("%d  %8.6f\r\n", i, aiOutData[i]);
        if (max < aiOutData[i])
        {
            count = i;
            max = aiOutData[i];
        }
    }
    printf("current number is %d\r\n", count);
}
```

八、完成心得体会

在完成过程中我们遇到了不少的困难与挑战，比如硬件配置上一开始配置出错，没有配置到 USART1 对应的重映射管脚，导致串口没有反应；编程上如何学习串口接收不定长数据的方式再加上利用 strtok 函数实现切割；模型训练上一开始训练效果不好，欠拟合下学习率不足，后来模型又做的过大，导致模型分析时候空间不足报错；在 MATLAB 实现过程中因为输出数据格式不正确导致 strtok 函数运行异常，串口卡死。可以说，在整个大作业过程中我们是不断地学习不断地自我修正。我们查阅了很多资料来完成我们整套所需的构思和编程。同时在这里我们还要感谢徐和根老师在我们完成过程中的悉心教导，老师教导帮助我们定位出错的位置，提供示例程序学习如何处理数据。

九、踩坑记录

①在 cubeMX 中，当添加 AI 软件包之后，此时时钟源和时钟树会强行修改并且报错，这是 bug，需要手动强行修改回去，之后直接生成工程代码编程即可。



②生成工程代码时堆栈大小尽可能设置大一些，防止可能存在的出错

Advanced Settings	
Linker Settings	
Minimum Heap Size	0x900
Minimum Stack Size	0x900
Thread-safe Settings	